

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 3

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

- Import Libraries: Import TensorFlow/Keras, Pandas, and Matplotlib for model creation, training, evaluation, and visualization.
- Load Dataset: Load the Fashion MNIST dataset containing 28×28 grayscale clothing images and display sample images with their class labels.
- Preprocess Data: Flatten each 28×28 image into 784 features and normalize pixel values from 0–255 to 0–1.
- Create Neural Network: Build a Sequential neural network with 128 and 64 ReLU hidden neurons and a 10-neuron Softmax output layer. Compile it using Adam and sparse categorical cross-entropy.
- Analyze Learning Rates: Train separate models using learning rates 0.1, 0.01, and 0.001 for 5 epochs. Evaluate and compare their test loss and accuracy.
- Plot Learning Rate Performance: Store the results in a DataFrame and plot Learning Rate vs Test Accuracy.
- Analyze Number of Epochs: Using a fixed learning rate of 0.001, train models for 5, 10, and 20 epochs and compare their test accuracy and loss.
- Plot Epoch Performance: Plot Number of Epochs vs Test Accuracy to observe the effect of training duration.
- Perform Forward and Backpropagation: During training, Keras performs forward propagation to generate predictions, calculates the loss, and uses backpropagation with the Adam optimizer to update network weights.
- Analyze Final Model: Train the model for 20 epochs and plot Training vs Validation Accuracy and Training vs Validation Loss to analyze model performance and possible overfitting.

Code:

```
1] # Importing the required libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd

2] (x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

print("Training data:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing data:", x_test.shape)
print("Testing labels:", y_test.shape)

class_names = [
    'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
    'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'
]

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title(class_names[y_train[i]])
    plt.axis('off')

plt.tight_layout()
plt.show()

3] # Preprocess the data and creating the neural network
x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0

print(x_train.shape)
print(x_test.shape)

# Creating the neural network
def create_model(lr):
    model = Sequential([
        Dense(128, activation='relu', input_shape=(784,)),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
```

```

    return model

4] learning_rates = [0.1, 0.01, 0.001]

results = []

for lr in learning_rates:

    print("Training with learning rate:", lr)

    model = create_model(lr)

    model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(x_test, y_test, verbose=0)

    results.append([lr, loss, acc])

5] # Displaying the learning-rate results
df = pd.DataFrame(
    results,
    columns=['Learning Rate', 'Loss', 'Accuracy']
)

print(df)

# plotting learning rate vs accuracy
plt.plot(
    df['Learning Rate'],
    df['Accuracy'],
    marker='o'
)

plt.xscale('log')
plt.xlabel('Learning Rate')
plt.ylabel('Test Accuracy')
plt.title('Effect of Learning Rate on Model Accuracy')
plt.grid()
plt.show()

6] # Testing with diff epochs and plotting an epoch vs accuracy graph
epochs_list = [5, 10, 20]
epoch_results = []

for ep in epochs_list:
    model = create_model(0.001)

    model.fit(
        x_train,
        y_train,

```

```

        validation_split=0.2,
        epochs=ep,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    epoch_results.append([ep, loss, acc])

edf = pd.DataFrame(
    epoch_results,
    columns=['Epochs', 'Loss', 'Accuracy']
)

print(edf)

plt.plot(edf['Epochs'], edf['Accuracy'], marker='o')
plt.xlabel('Number of Epochs')
plt.ylabel('Test Accuracy')
plt.title('Effect of Epochs on Model Accuracy')
plt.grid()
plt.show()

7] model = create_model(0.001)

history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=128
)

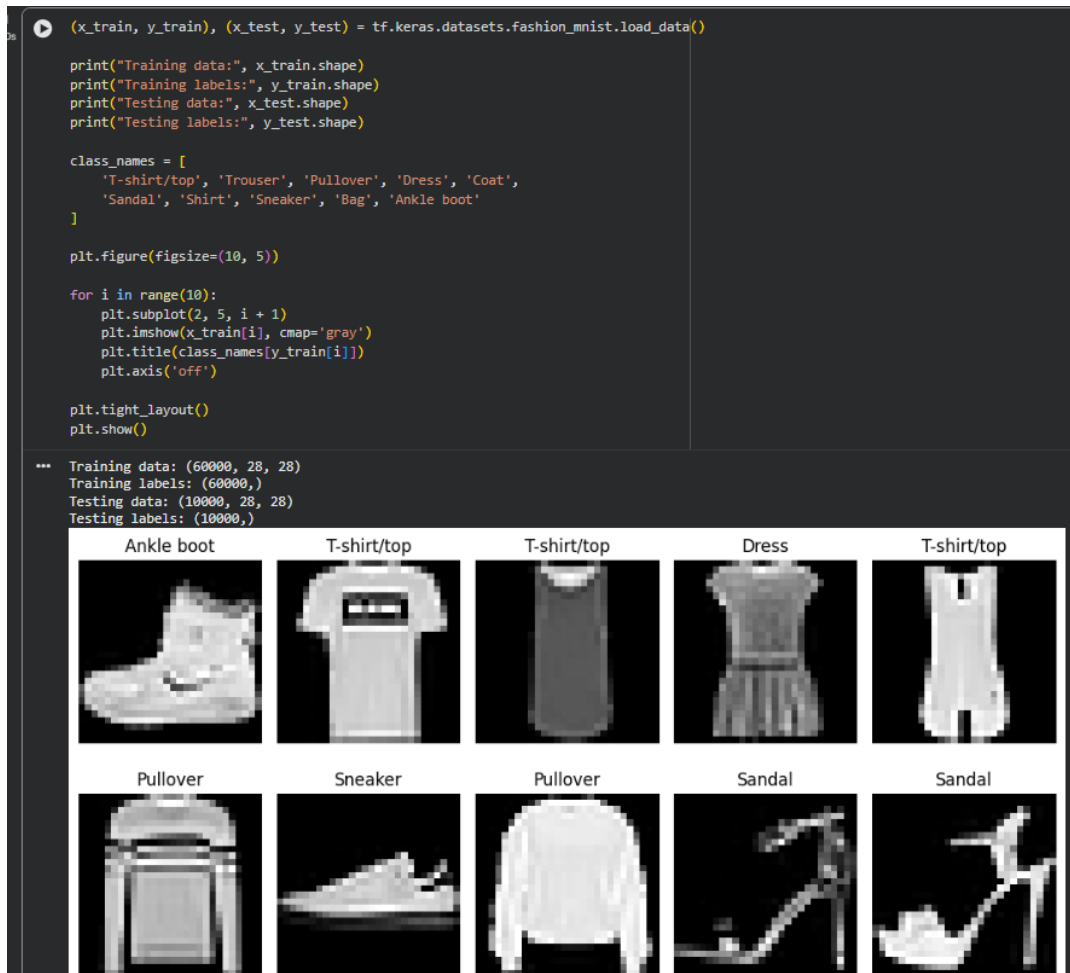
# Accuracy graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend(['Train', 'Validation'])
plt.grid()
plt.show()

# Loss graph
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend(['Train', 'Validation'])
plt.grid()
plt.show()

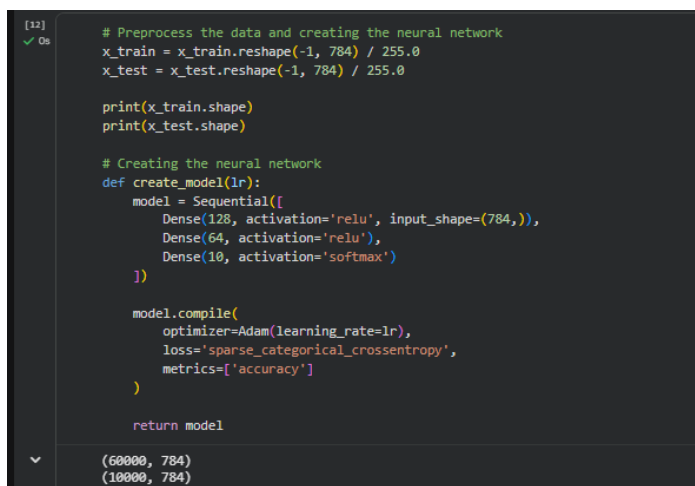
```

Output:

1]



2]



3]

```

learning_rates = [0.1, 0.01, 0.001]
results = []

for lr in learning_rates:
    print("Training with learning rate:", lr)

    model = create_model(lr)

    model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=128,
        verbose=0
    )

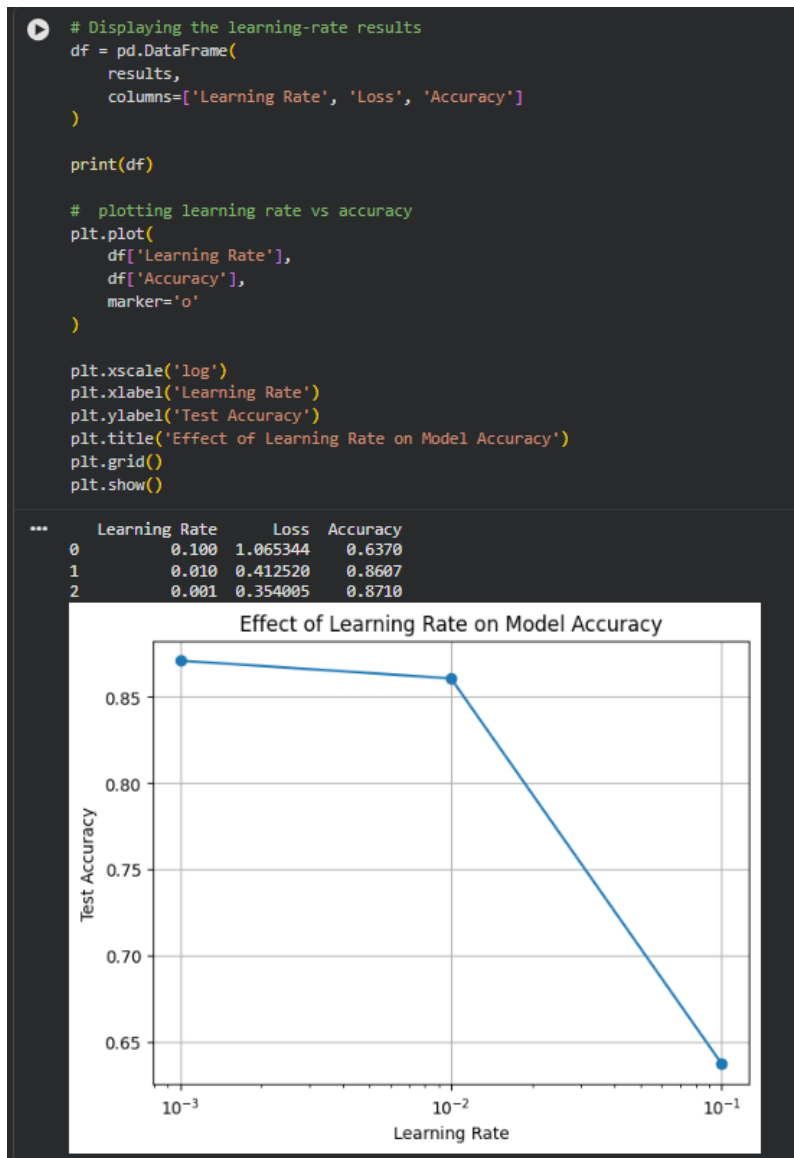
    loss, acc = model.evaluate(x_test, y_test, verbose=0)

    results.append((lr, loss, acc))

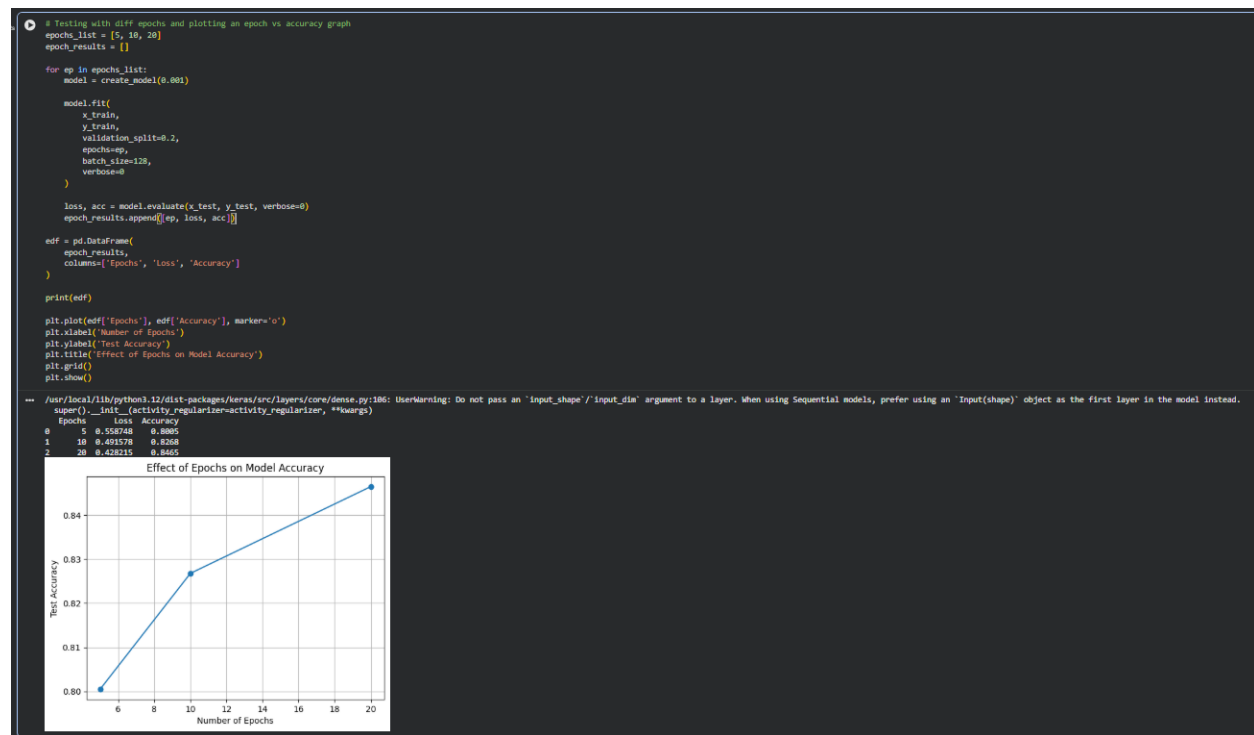
... Training with learning rate: 0.1
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training with learning rate: 0.01
Training with learning rate: 0.001

```

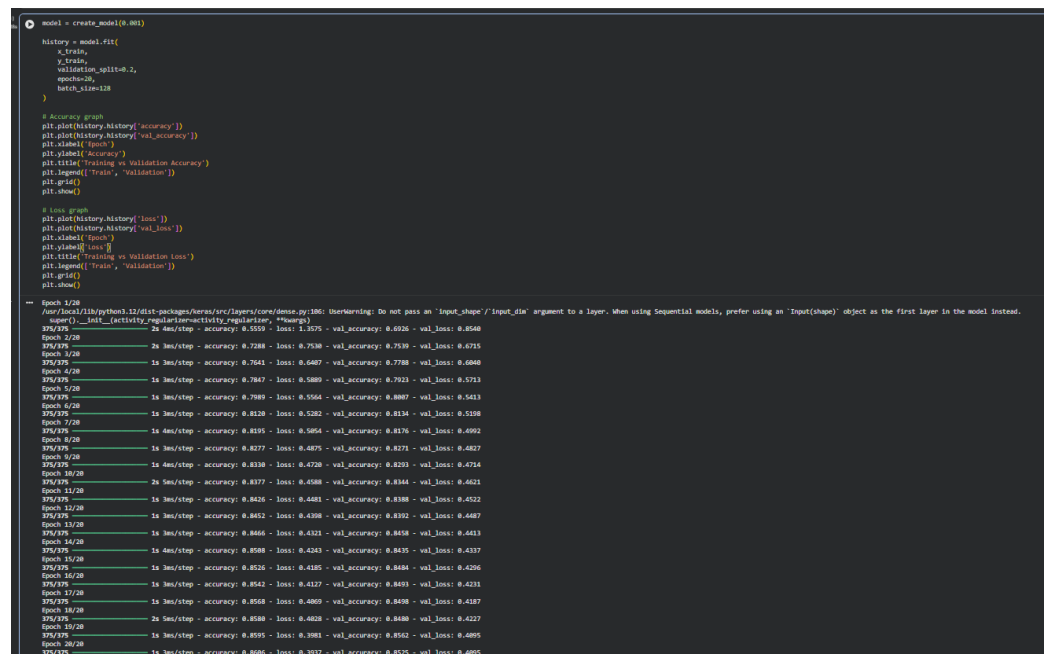
4]



5]



6]



7]

